

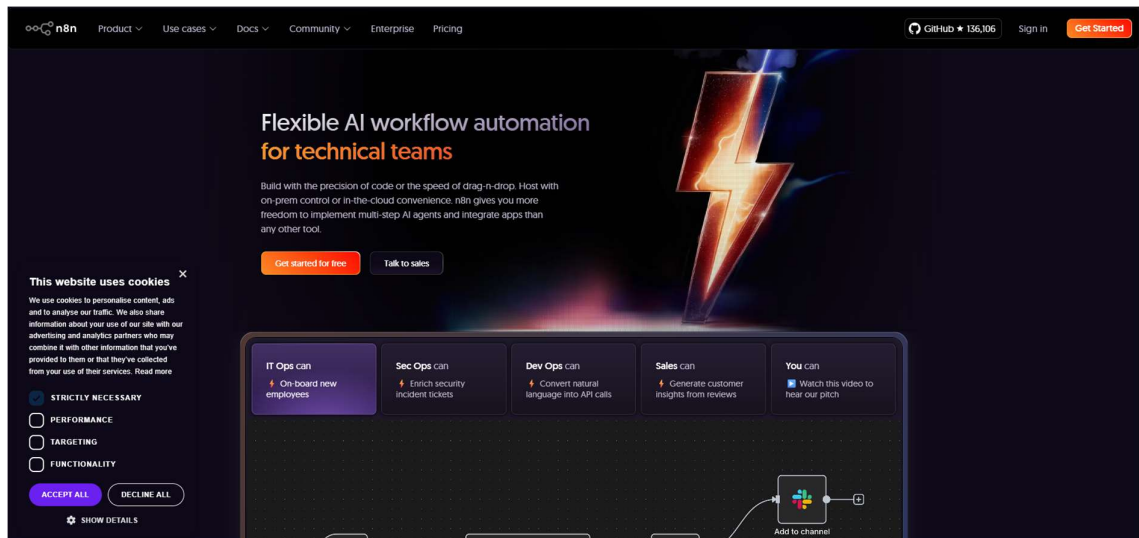
CN7050- Intelligent Systems Module

Develop n8n workflow that predicts diabetes using the K-Nearest Neighbours (KNN) algorithm.

Practice Session for N8N to design an automation workflow

I. Access n8n

- Go to your navigator and tap : <https://n8n.io/>
- Click to GET STARTED FREE



- Enter your email address...Etc and clique get started free 14-day trial (you will have 14 days for free)

Start automating today

Full name

Company email

Confirm email address

Password

Account name

☐ Keep up with our constant improvements through our newsletter

 Succès !

 **CLOUDFLARE**
Confidentialité • Conditions

Start free 14-day trial

By continuing, you agree to our [terms and conditions](#) and [Privacy policy](#).

- Select the option G I'm not using n8n for work and click Ok

Hi there, Anas

Answer these questions to get the most out of your n8n trial

1

What is the size of your company?*

This helps us to customize your experience in n8n

A

Only me

B

<20

C

20-99

D

100-499

E

500-999

F

1000+

G

I'm not using n8n for work

OK

- Choose one of these choices that you feel comfortable to do

2→ Which of these do you feel comfortable doing?

Choose as many as you like

- ☐ A Writing JavaScript functions
- ☐ B Using SQL to query a database
- ☐ C Configuring API authentication
- ☐ D Using regex for text extraction
- ☐ E Setting up a webhook

OK

- Skip the step

Invite team members to your workspace

Collaborate on workflows and share app credentials with workspace users

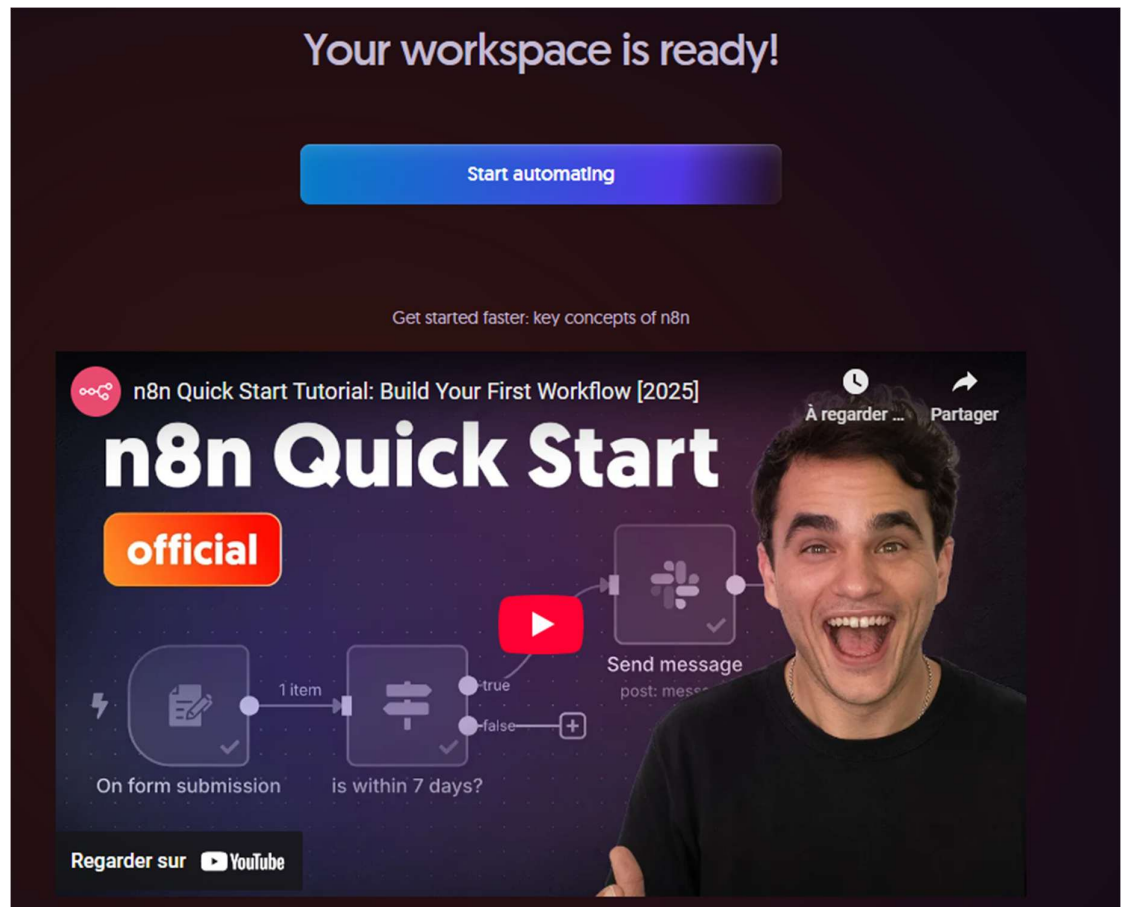
Email

name@email.com

Add another user +

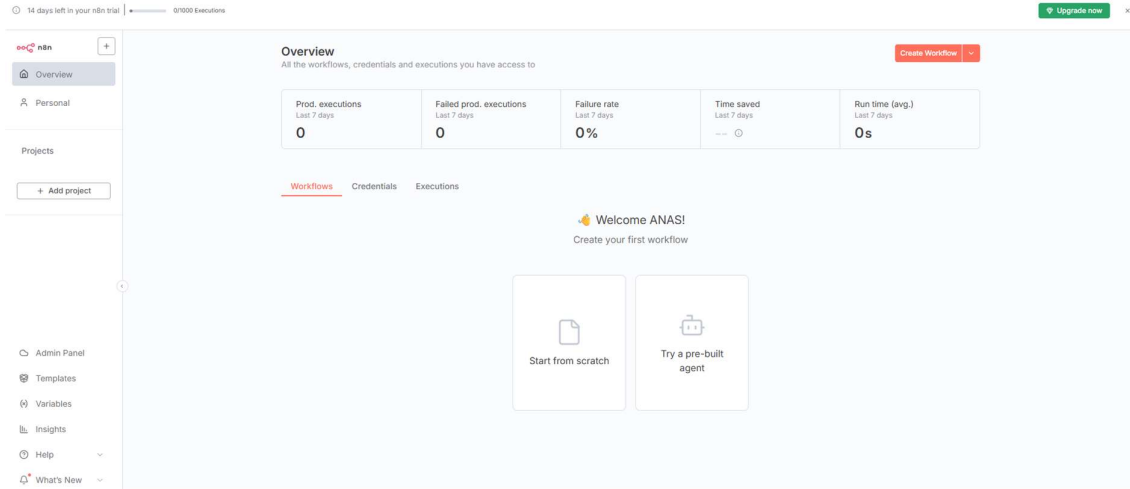
Skip Invite

- Now your workspace is ready, click start automating



II. Start the LAB

- This the home page of n8n when we will create the workflows



Overview

You'll build this workflow:

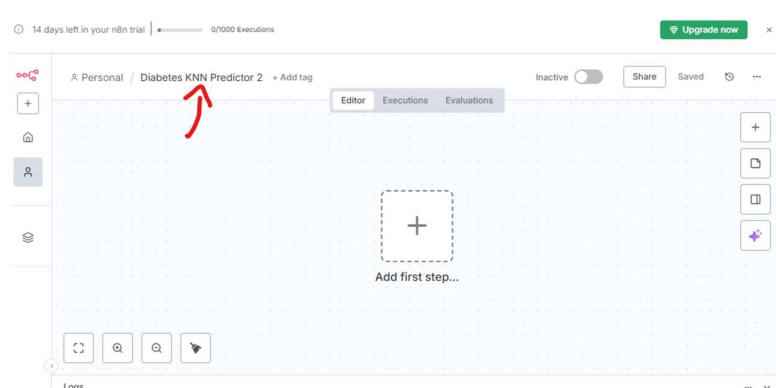
Webhook (POST file) → Extract From File (CSV → rows) → Code: Prepare Data → Code: Split & Scale → Code: Train & Evaluate (KNN)

0) Get n8n running

- **Easiest:** n8n Cloud → create a workspace → open the Editor.

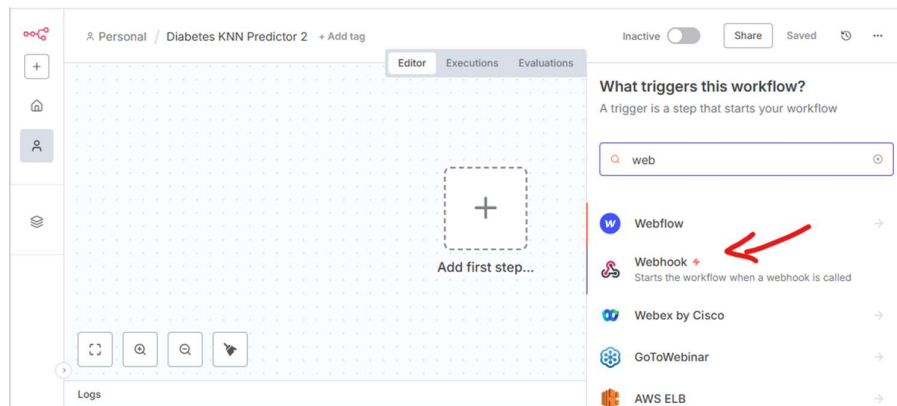
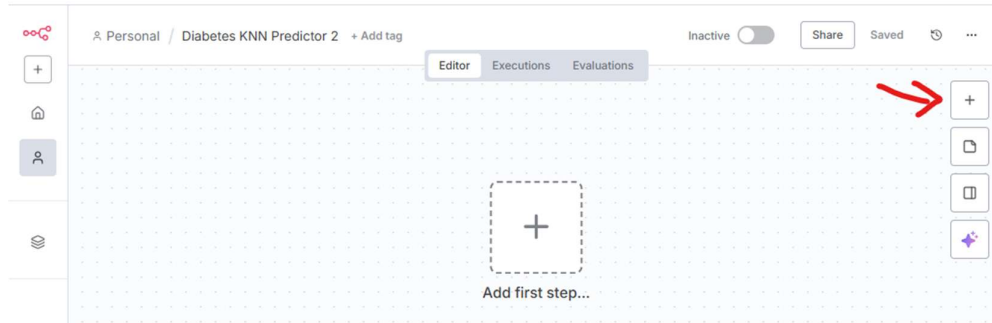
1) Create a new workflow

- Click + **New** → name it "Diabetes KNN Predictor 2".



2) Create the Webhook (to receive your CSV file)

1. In n8n Cloud, click + **New workflow**.
2. Add **Webhook** node.



3. Configure:

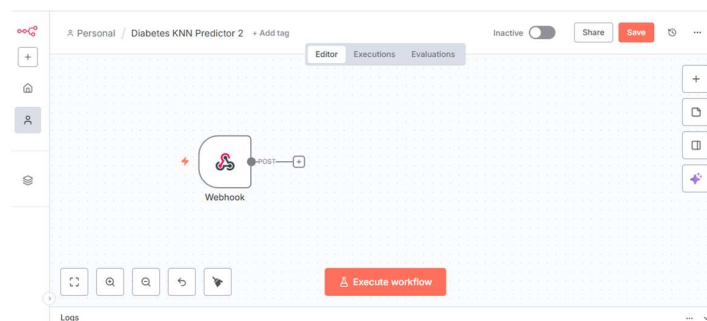
- **HTTP Method:** POST
- **Path:** diabetes-upload (so the URL ends with /diabetes-upload)
- **Response:** Last node
- **Response Code:** 200
- Leave other defaults.

The screenshot shows the configuration panel for a 'Webhook' node. At the top, there's a 'Listen for test event' button. Below it are tabs for 'Parameters' (selected) and 'Settings', along with a 'Docs' link. Under 'Webhook URLs', there are 'Test URL' and 'Production URL' buttons. The 'Test URL' is active, showing a POST method and the URL 'https://ueltest.app.n8n.cloud/webhook-test/diabetes-upload'. Below this, several configuration fields are visible: 'HTTP Method' set to POST, 'Path' set to diabetes-upload, 'Authentication' set to None, 'Respond' set to 'When Last Node Finishes', 'Response Data' set to 'First Entry JSON', and 'Response Code' set to 200.

You'll see two URLs at the top of the Webhook node:

- **Test URL** (works while the editor is in “Test” mode)
- **Production URL** (works after you activate the workflow)

For now, keep the workflow in the editor and use the **Test URL**



3) Parse the uploaded CSV

Add **Extract From File** node and connect it after **Webhook** (from the **right edge** main port of Webhook → left of Extract).

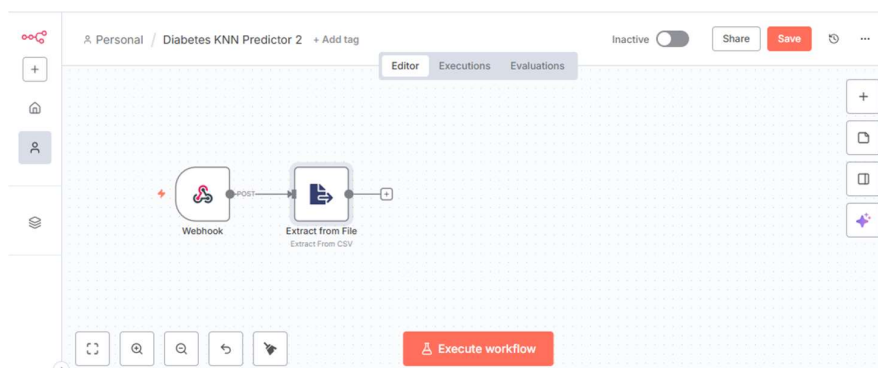
- **Operation:** Extract from CSV
- **Input Binary Field:** file

Why file? When you upload via multipart/form-data, the form field name becomes the binary property. We'll upload using the key **file** so this matches.

- **Options:**
 - **First Row Is Header:** ON
 - **Delimiter:** ,
 - **Encoding:** utf8
 - **Skip Empty Lines:** ON (optional)

The screenshot shows the configuration panel for the 'Extract from File' node. It has three tabs: 'Parameters' (selected), 'Settings', and 'Docs'. Under 'Parameters', the 'Operation' dropdown is set to 'Extract From CSV'. The 'Input Binary Field' text input is set to 'file'. Below this, a note states: 'The name of the input field containing the file data to be processed'. Under the 'Options' section, the 'Delimiter' is set to ',', 'Encoding' is set to 'UTF-8', and the 'Header Row' toggle is turned on. An 'Add option' button is at the bottom.

When this runs, you should get **many output items** (one per row in the CSV).



4) Prepare data (auto-detect features/label, numeric cast)

Add a **Code** node called **Prepare Data (auto-cast)** after **Extract from File** and paste the code existing in the file below (script1)



Script 1.txt

Code in JavaScript

Execute step

ParametersSettingsDocs

Mode

Run Once for All Items

Language

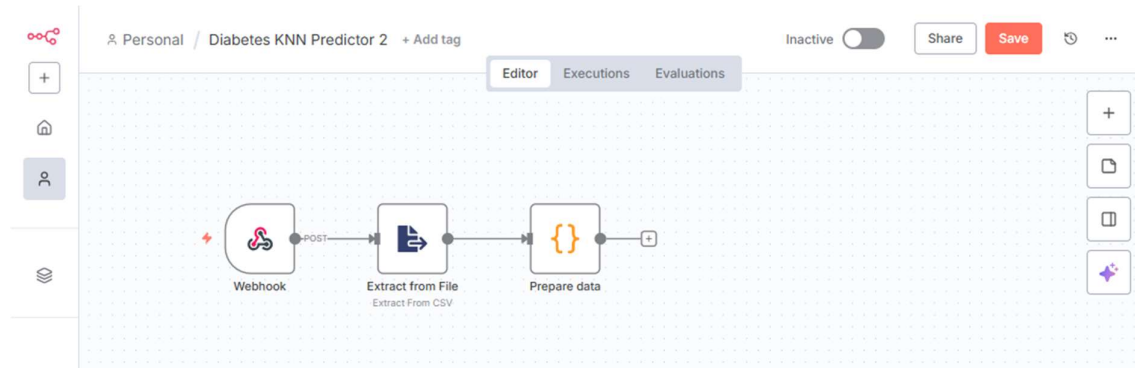
JavaScript

JavaScript

CodeAsk AI

```
27 // Build matrices
28 const X = rows.map(r => featureNames.map(f => toNum(r[f])));
29 const y = rows.map(r => toNum(r[labelName]));
30
31 if (!X.length || !X[0].length) {
32   throw new Error("No features detected. Check your CSV content.");
33 }
34
35 return [{
36   json: {
37     featureNames,
38     labelName,
39     X,
40     y,
41     nSamples: X.length,
42     nFeatures: featureNames.length
43   }
44 }];
45
```

Type \$ for a list of special vars/methods. Debug by using console.log() statements and viewing their output in the browser console.



5) Split & Scale (80/20 split, z-score on TRAIN only)

Add a **Code** node called **Split & Scale** after the previous one and paste the code existing in the file below (script 2) :



Script 2.txt

Split & Scale

Execute step

Parameters

Settings

Docs

Mode

Run Once for All Items

Language

JavaScript

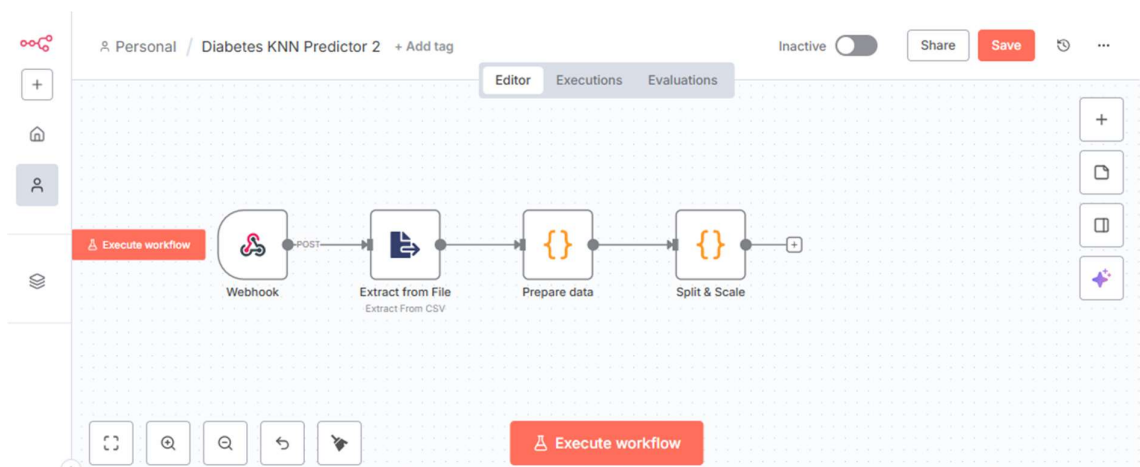
JavaScript

Code

Ask AI

```
1 // SPLIT & SCALE
2 // - Deterministic shuffle (seed=42)
3 // - 80/20 split
4 // - Fit z-score scaler on TRAIN only; apply to both TRAIN and TEST
5
6 const { featureNames, labelName, X, y } = items[0].json;
7 const testRatio = 0.2;
8 const seed = 42;
9
10 // Deterministic RNG
11 let s = seed;
12 const rng = () => (s = (s*1664525 + 1013904223) % 4294967296) / 4294967296;
13
14 // Shuffle indices
15 const idx = Array.from({ length: X.length }, (_, i) => i);
16 for (let i = idx.length - 1; i > 0; i--) {
17   const j = Math.floor(rng() * (i + 1));
18   [idx[i], idx[j]] = [idx[j], idx[i]];
19 }
```

Type `$` for a list of **special vars/methods**. Debug by using `console.log()` statements and viewing their output in the browser console.



6) Train & Evaluate (KNN with grid-search k)

Add a **Code** node called **Train & Evaluate (KNN)** and paste the code existing in the file below (script 3) :



Script 3.txt

Code in JavaScript

Execute step

ParametersSettingsDocs

Mode

Run Once for All Items

Language

JavaScript

JavaScript

CodeAsk AI

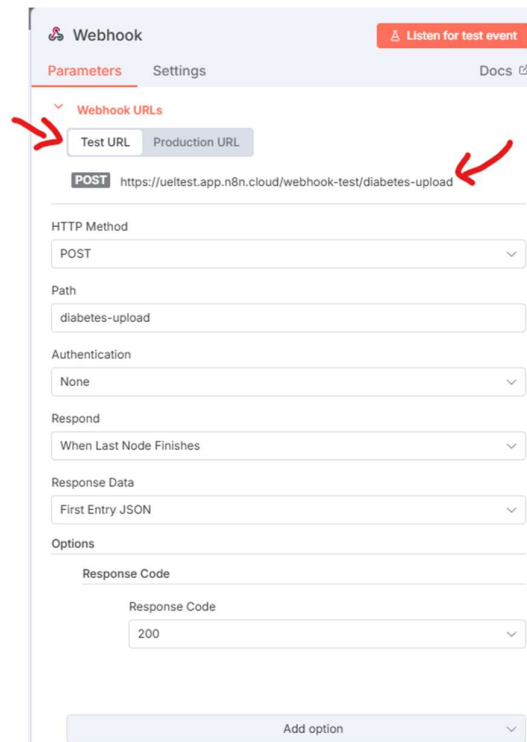
```
1 // TRAIN & EVALUATE (KNN)
2 // - Grid search k
3 // - Report best k, test accuracy, confusion matrix
4 // - Also return train accuracy (sanity check)
5
6 const { featureNames, train, test } = items[0].json;
7 const { X: Xt, y: yt } = train;
8 const { X: Xv, y: yv } = test;
9
10 function dist(a,b){ let s=0; for(let i=0;i<a.length;i++) s += (a[i]-b[i])**2;
11   return Math.sqrt(s); }
12 function knnPredict(trainX, trainY, xNew, k=3){
13   const dists = trainX.map((row, i) => ({ i, d: dist(row, xNew) }));
14   dists.sort((a,b) => a.d - b.d);
15   const knn = dists.slice(0, k);
16   const counts = new Map();
17   for (const n of knn) {
18     const cls = trainY[n.i];
19     counts.set(cls, (counts.get(cls) || 0) + 1);
20   }
21   const bestCls = [...counts.entries()].sort((a,b) => b[1] - a[1])[0][0];
22   return bestCls;
23 }
```

Type \$ for a list of special vars/methods. Debug by using console.log() statements and viewing their output in the browser console.



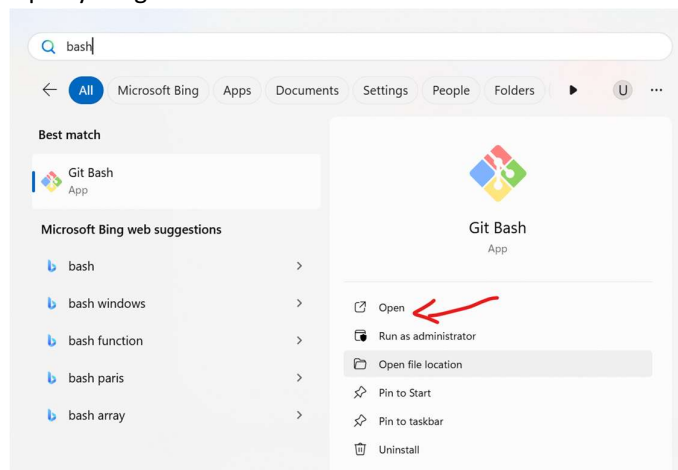
7) How to upload your CSV to the Webhook

While the editor is open, click the Webhook node and copy the **Test URL** (it contains /webhook-test/diabetes-upload).

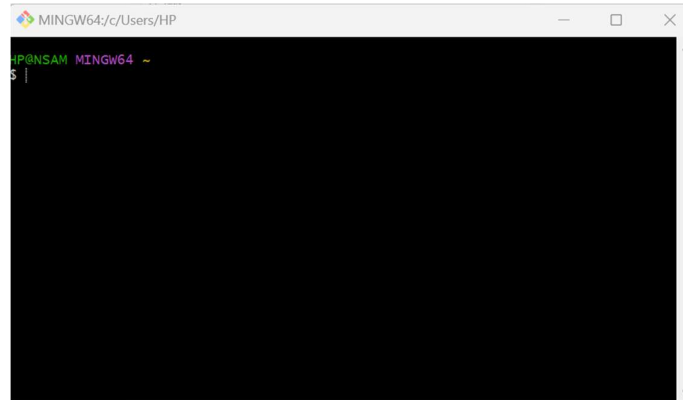


— curl (quickest)

- Open your git bash

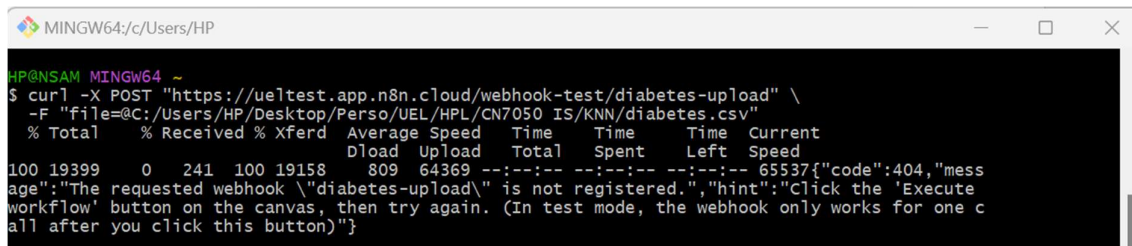


- Open your git bash



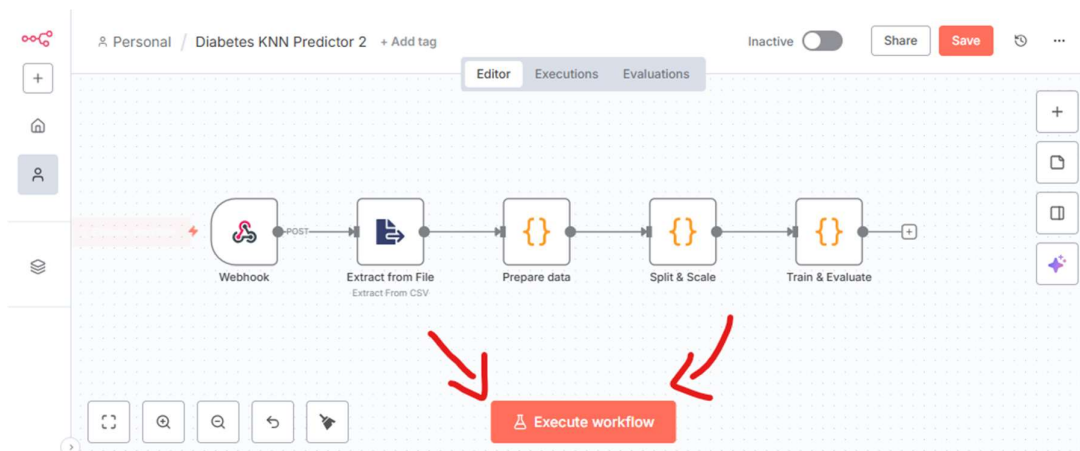
- Execute this in your git bash

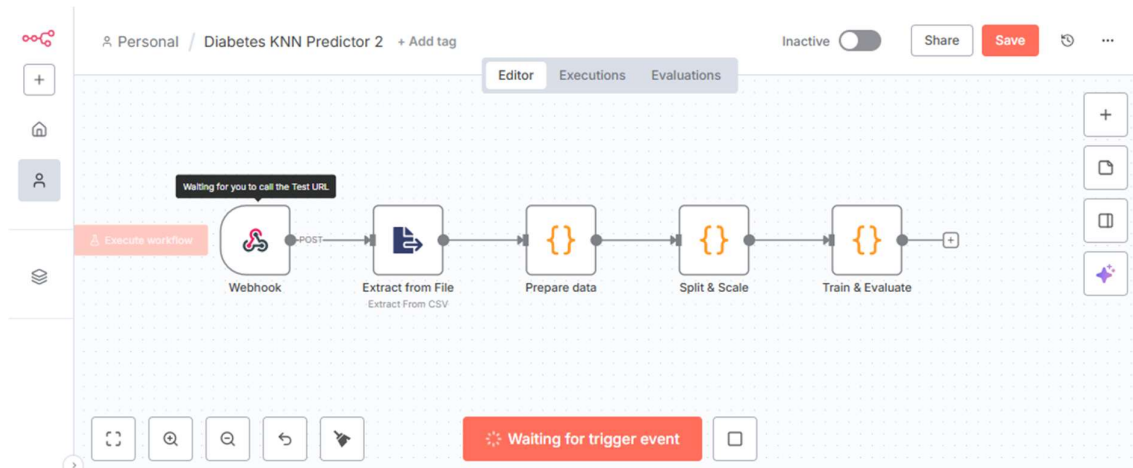
```
curl -X POST "<YOUR_TEST_URL>" \
-F "file=@/path/to/diabetes.csv"
```
- Replace <YOUR_TEST_URL> with the exact Test URL.
- Replace the path after @ with your local CSV path.



8) Execute workflow to test it

- Execute the workflow

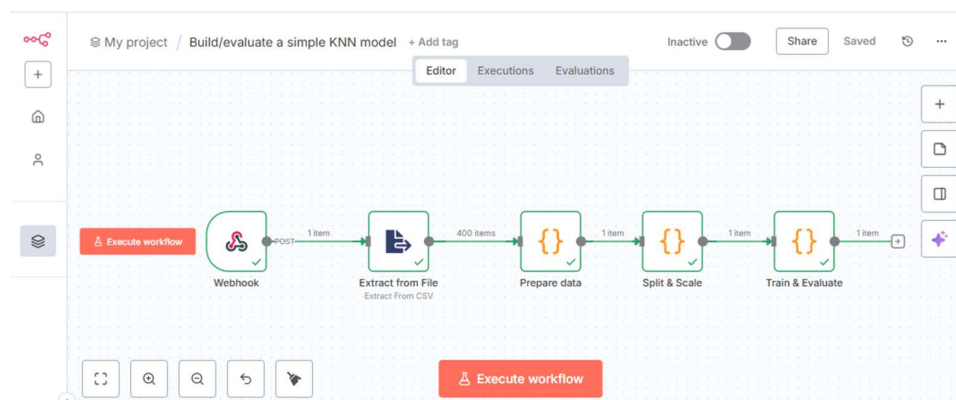




- You need now to execute the command in you git bash

```
curl -X POST "https://ueltest.app.n8n.cloud/webhook-test/diabetes-upload" \
-F "file=@C:/Users/HP/Desktop/Perso/UEL/HPL/CN7050 IS/KNN/diabetes.csv"
```

```
MINGW64/c/Users/HP
100 19399 0 241 100 19158 809 64369 --:--:-- --:--:-- --:--:-- 65537f"code":404,"message":"The requested webhook \"diabetes-upload\" is not registered.,\"hint\":\"Click the 'Execute workflow' button on the canvas, then try again. (In test mode, the webhook only works for one call after you click this button)"}
HP@NSAM MINGW64 ~
$ AC
HP@NSAM MINGW64 ~
$ ^[[200-curl -X POST "https://ueltest.app.n8n.cloud/webhook/diabetes-upload" \
-F "file=@C:/Users/HP/Desktop/Perso/UEL/HPL/CN7050 IS/KNN/diabetes.csv"
bash: $'\E[200-curl': command not found
HP@NSAM MINGW64 ~
$ curl -X POST "https://ueltest.app.n8n.cloud/webhook-test/diabetes-upload" \
-F "file=@C:/Users/HP/Desktop/Perso/UEL/HPL/CN7050 IS/KNN/diabetes.csv"
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 19301 0 143 100 19158 59 7951 0:00:02 0:00:02 --:--:-- 8022f"model":"KNN","nTrain":320,"nTest":80,"bestK":7,"testAccuracy":0.75,"trainAccuracy":0.790625,"labels":[0,2],"confusionMatrix":[[55,3],[17,5]]}
HP@NSAM MINGW64 ~
$
```



9) Check the outputs for each component

- Webhook

The screenshot shows a workflow diagram with five components: Webhook, Extract from File, Prepare data, Split & Scale, and Train & Evaluate. The Webhook component is selected, and its output is displayed in the 'OUTPUT' section. The output is a file named 'diabetes.csv' with a size of 18.9 kB. The 'Logs' section shows 'Success in 30.71s'.

Workflow Diagram:

```
graph LR; Webhook[Webhook] -- "1 item" --> Extract[Extract from File]; Extract -- "400 items" --> Prepare[Prepare data]; Prepare -- "1 item" --> Split[Split & Scale]; Split -- "1 item" --> Train[Train & Evaluate]; Train -- "1 item" --> End[+];
```

OUTPUT

file

File Name: diabetes.csv
File Extension: bin
Mime Type: application/octet-stream
File Size: 18.9 kB

Download

Logs Clear execution Webhook Success in 30.71s

- Extract from File

The screenshot shows the same workflow diagram as above, but with the 'Extract from File' component selected. The 'INPUT' section shows the file 'diabetes.csv' with a size of 18.9 kB. The 'OUTPUT' section shows a list of features: HighBP, HighChol, CholCheck, BMI, Smoker, Stroke, HeartDiseaseorAttack, PhysActivity, Fruits, and Veggies, each with a value of 1 or 0. The 'Logs' section shows 'Success in 30.71s'.

Workflow Diagram:

```
graph LR; Webhook[Webhook] -- "1 item" --> Extract[Extract from File]; Extract -- "400 items" --> Prepare[Prepare data]; Prepare -- "1 item" --> Split[Split & Scale]; Split -- "1 item" --> Train[Train & Evaluate]; Train -- "1 item" --> End[+];
```

INPUT

file

File Name: diabetes.csv
File Extension: bin
Mime Type: application/octet-stream
File Size: 18.9 kB

Download

OUTPUT

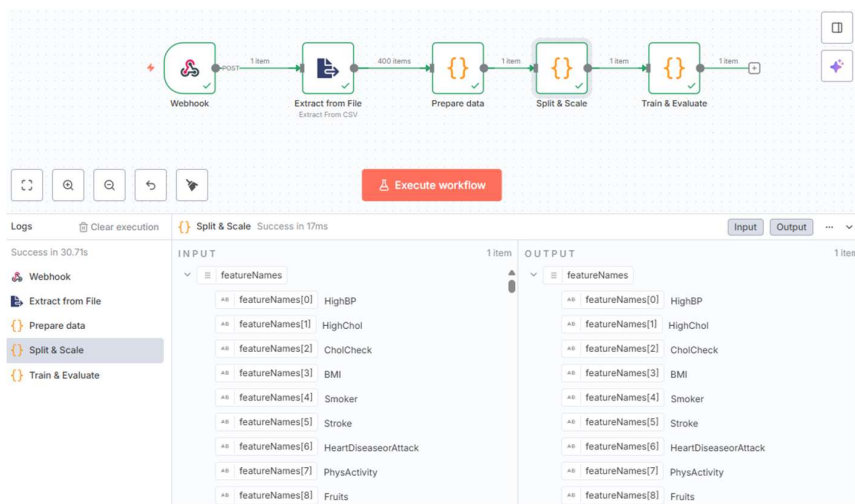
Feature	Value
HighBP	1
HighChol	1
CholCheck	1
BMI	40
Smoker	1
Stroke	0
HeartDiseaseorAttack	0
PhysActivity	0
Fruits	0
Veggies	1

Logs Clear execution Extract from File Success in 30.71s

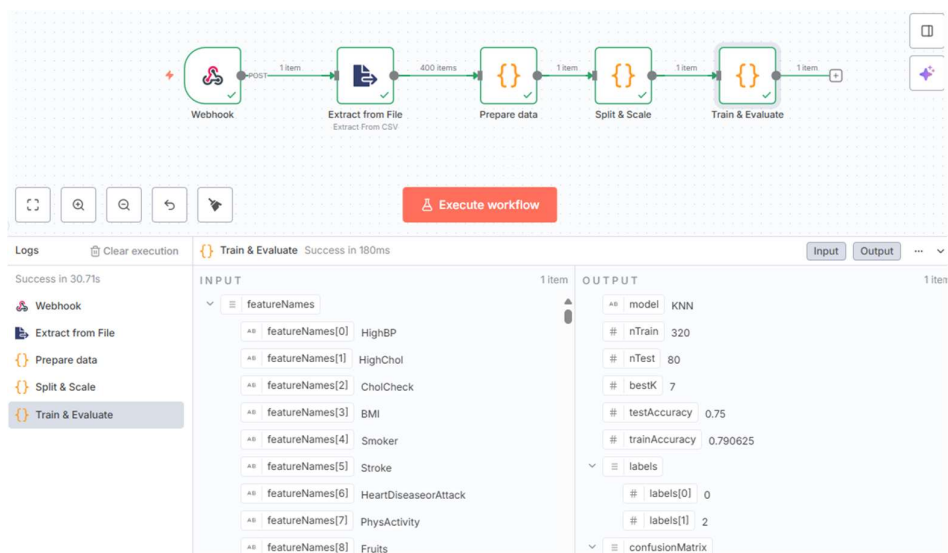
- Prepare data



- Split & Scale



- Train & Evaluate



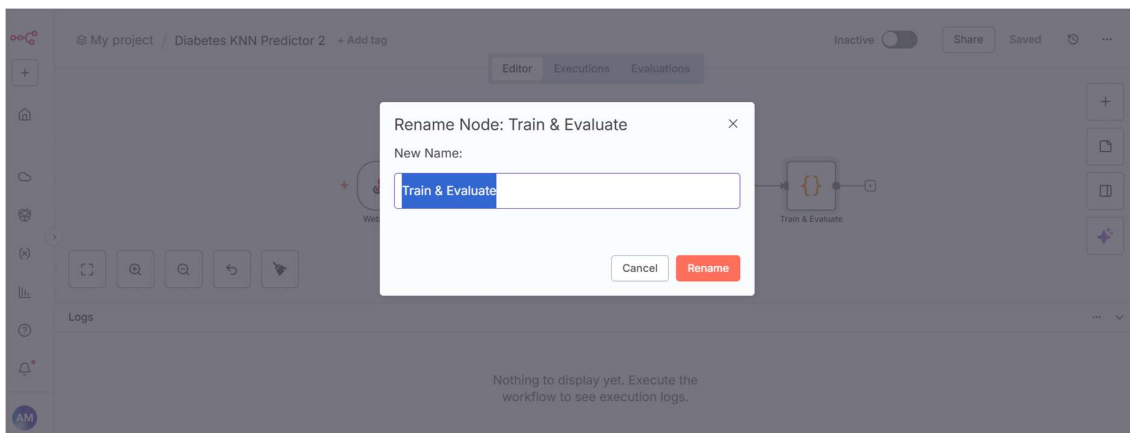
10) Test the workflow

1. Part A — “TRAIN” endpoint (upload CSV once)

Now we have a workflow like that:

Webhook (POST /diabetes-upload) → Extract From File (CSV) → Prepare Data → Split & Scale → Train & Evaluate (KNN)

We will replace the last node with this “Store Model & Report”, use the script below.



- Update the script, past the script 4 on the code area.



Script 4.txt

Store Model & Report

Execute step

Parameters

Settings

Docs

Mode

Run Once for All Items

Language

JavaScript

JavaScript

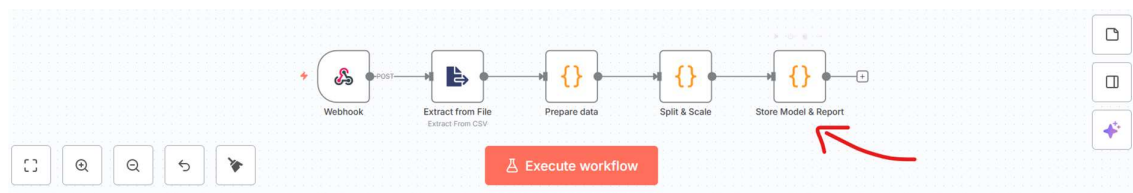
Code

Ask AI

```

6  * test: { X, y, size },
7  * scaler: { mean, stdev }
8  * }
9  * ACTIONS:
10 * - Train a small KNN (grid search k)
11 * - Save model pieces to workflow static data (so another webhook can reuse them)
12 * OUTPUT:
13 * - JSON with metrics (bestK, testAccuracy...) to return to caller
14 */
15
16 const { featureNames, labelName, train, test, scaler } = items[0].json;

```



- How to call TRAIN (you already did):
 - Keep your current Webhook node (POST /diabetes-upload) in Active mode or Test mode.
 - Upload the CSV like before:
 - `curl -X POST "https://ueltest.app.n8n.cloud/webhook-test/diabetes-upload" \`
 - `-F file=@C:/Users/HP/Desktop/Perso/UEL/HPL/CN7050 IS/KNN/diabetes.csv`
- Response returns metrics, and the **model is stored** in the workflow's static data.

My project / Diabetes KNN Predictor 2 + Add tag

Inactive

Share

Saved

...

Editor

Executions

Evaluations

Webhook

Extract from File

Prepare data

Split & Scale

Store Model & Report

Execute workflow

Logs

Clear execution

Store Model & Report Success in 180ms

Input

Output

Success in 10.292s

Extract from File

Prepare data

Split & Scale

Store Model & Report

INPUT

1 item

featureNames

HighBP

HighChol

CholCheck

RMI

OUTPUT

1 item

summary

model=KNN | train=320 | test=80 | k=7 | acc_test=0.750 | acc_train=0.791

model

KNN

bestK

7

nTrain

320

nTest

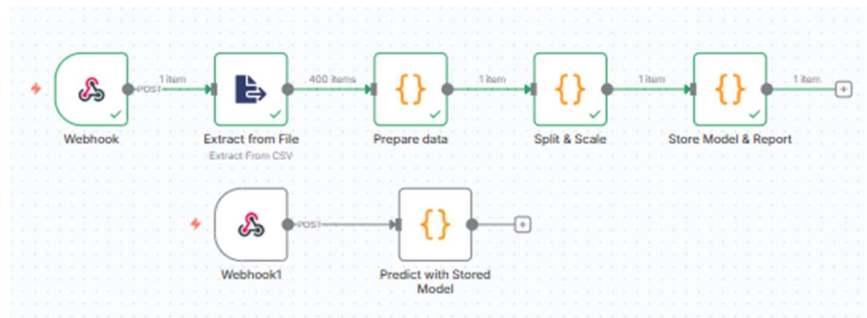
80

2. Part B — “PREDICT” endpoint (send features → get answer)

Add a new **Webhook** node:

- **HTTP Method:** POST
- **Path:** diabetes-predict
- **Response:** Last node
- **(Activate the workflow to use the production URL.)**

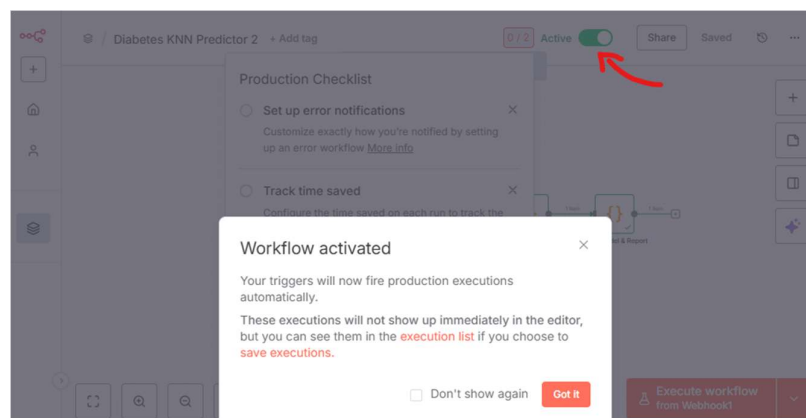
Connect this Webhook to the **Code** node named **Predict with Stored Model** and paste the script 5 on the code area:




Script 5.txt

- **How to actually predict**

1. Make sure you’ve **trained** at least once:
 - POST your CSV to the **training Webhook** (/diabetes-upload).
 - Check that “Store Model & Report” succeeded (green, output shows metrics).
2. Activate workflow if you want to call it without “Execute workflow”.



3. POST JSON to the prediction URL (lower Webhook):

```
curl -X POST "https://ueltest.app.n8n.cloud/webhook/diabetes-predict" \
```

```
-H "Content-Type: application/json" \
```

```
-d
```

```
{'HighBP':1,'HighChol':1,'CholCheck':1,'BMI':30,'Smoker':0,'Stroke':0,'HeartDiseaseorAttac
k':0,'PhysActivity':1,'Fruits':1,'Veggies':1,'HvyAlcoholConsump':0,'AnyHealthcare':1,'NoDoc
bcCost':0,'GenHlth':3,'MentHlth':2,'PhysHlth':1,'DiffWalk':0,'Sex':0,'Age':9,'Education':4,'I
ncome':6}'
```

```
HP@NSAM MINGW64 ~
$ curl -X POST "https://ueltest.app.n8n.cloud/webhook/diabetes-upload" \
-F "file=@C:/Users/HP/Desktop/Perso/UEL/HPL/CN7050 IS/KNN/diabetes.csv"
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 19386 0 228 100 19158 80 6753 0:00:02 0:00:02 --:--:-- 6838{"summary":"model=KNN | train=320
| test=80 | k=7 | acc_test=0.750 | acc_train=0.791","model":"KNN","bestK":7,"nTrain":320,"nTest":80,"testAccur
acy":0.75,"trainAccuracy":0.790625,"labels":[0,2],"confusionMatrix":[[55,3],[17,5]]}
```

Upload csv file using the production URL

```
HP@NSAM MINGW64 ~
$ curl -X POST "https://ueltest.app.n8n.cloud/webhook/diabetes-predict" \
-H "Content-Type: application/json" \
-d '{"HighBP":1,"HighChol":1,"CholCheck":1,"BMI":30,"Smoker":0,"Stroke":0,"HeartDiseaseorAttack":0,"PhysActivity":1,"Fr
uits":1,"Veggies":1,"HvyAlcoholConsump":0,"AnyHealthcare":1,"NoDocbcCost":0,"GenHlth":3,"MentHlth":2,"PhysHlth":1,"Diffwa
lk":0,"Sex":0,"Age":9,"Education":4,"Income":6}'
% Total % Received % Xferd Average Speed Time Time Time Current
Dload Upload Total Spent Left Speed
100 1754 0 1471 100 283 689 132 0:00:02 0:00:02 --:--:-- 823{"ok":true,"usedK":7,"featuresOrder":["High
BP","HighChol","CholCheck","BMI","Smoker","Stroke","HeartDiseaseorAttack","PhysActivity","Fruits","Veggies","HvyAlcoholCo
nsump","AnyHealthcare","NoDocbcCost","GenHlth","MentHlth","PhysHlth","DiffWalk","Sex","Age","Education","Income"],"input"
":{"headers":{"host":"ueltest.app.n8n.cloud","user-agent":"curl/8.7.1","content-length":"283","accept":"*/","accept-encod
ing":"gzip, br","cdn-loop":"cloudflare; loops=1; subreqs=1","cf-connecting-ip":"2a00:23c7:8e06:4801:39b2:42bc:dcbe:f7e7",
"cf-ew-via":"15","cf-ipcountry":"GB","cf-ray":"97e8575906b4fd92-LHR","cf-visitor":{"scheme":"https"},"cf-worker":"n
8n.cloud"},"content-type":"application/json","x-forwarded-for":"2a00:23c7:8e06:4801:39b2:42bc:dcbe:f7e7","x-forwarded-host":"ueltest.app.n8n.cloud","x-forwarded-port":"443","x-forwarded-proto":"https","x-forwarded-server":"traefik-prod-users-gwc-30-6d57d45df-7fwhs","x-is-trusted":"yes","x-real-ip":"2a00:23c7:8e06:4801:39b2:42bc:dcbe:f7e7"},"params":{"query":{"},"body":{"HighBP":1,"HighChol":1,"CholCheck":1,"BMI":30,"Smoker":0,"Stroke":0,"HeartDiseaseorAttack":0,"PhysActivity":1,"Fruits":1,"Veggies":1,"HvyAlcoholConsump":0,"AnyHealthcare":1,"NoDocbcCost":0,"GenHlth":3,"MentHlth":2,"PhysHlth":1,"DiffWalk":0,"Sex":0,"Age":9,"Education":4,"Income":6},"webhookUrl":"https://ueltest.app.n8n.cloud/webhook/diabetes-predict","executionMode":"production","prediction":0,"predictionLabel":"Diabetes: NO (0)"}}
```

Upload the example to get the prediction

The screenshot shows the n8n workflow editor for a project named "Diabetes KNN Predictor 2". The workflow consists of several steps: Webhook, Extract from File, Prepare data, Split & Scale, Store Model & Report, and Predict with Stored Model. The "Predict with Stored Model" step is highlighted, showing its output in the "Log" panel. The output is a JSON object with the following structure:

```
{
  "prediction": 0,
  "predictionLabel": "Diabetes: NO (0)"
}
```

Red arrows point to the "prediction" and "predictionLabel" fields in the output. The "Log" panel also shows the input headers and body for the webhook call.

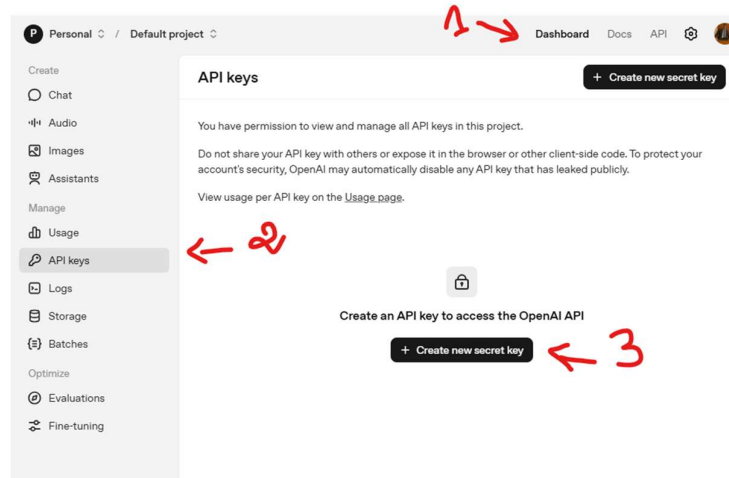
11) Common pitfalls (and fixes)

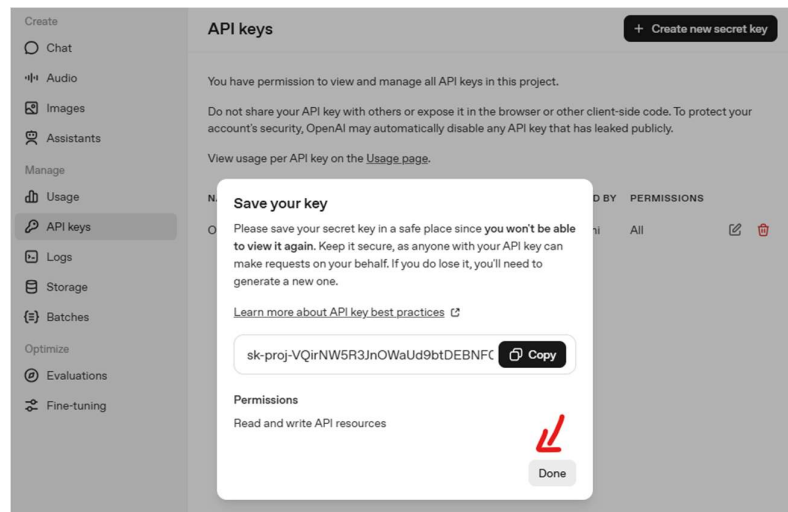
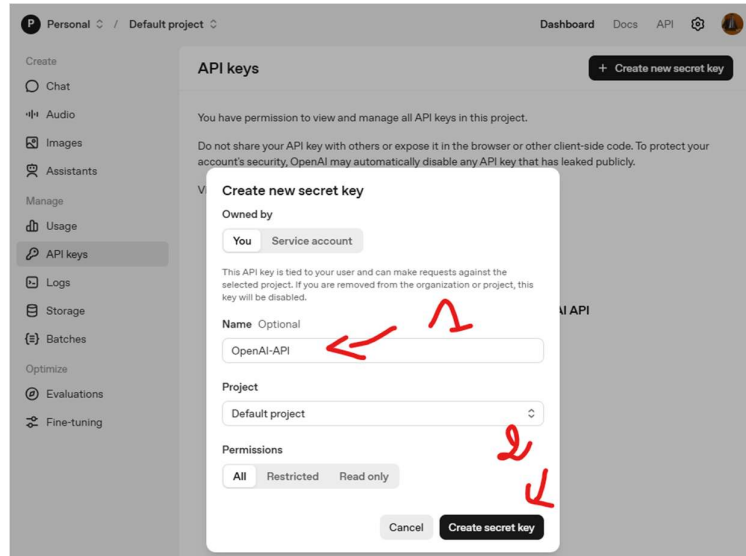
- **Webhook says 404**
Use the **Test URL** while the editor is in Test mode. After you **Activate**, use the **Production URL**.
- **Extract From File shows “No output data returned”**
Ensure your upload uses **form-data** with key `file`. In **Extract From File**, **Input Binary Field** = `file` (exact match). Also verify the CSV indeed has rows (open it locally).
- **Nodes don’t turn green**
Make sure connections are from the **right edge** (main output), not the tiny **bottom** error port.
- **Label name mismatch**
Code expects a column named `diabetes` (any casing). If yours is different, edit the detection in **Prepare Data** (set `const labelName = 'YourColumn'`).
- **Large CSVs**
n8n Cloud can handle sizeable files, but if it’s huge, test with a smaller subset first.

12) ADD an OpenAI “doctor-style” explainer to your *prediction* branch in n8n Cloud.

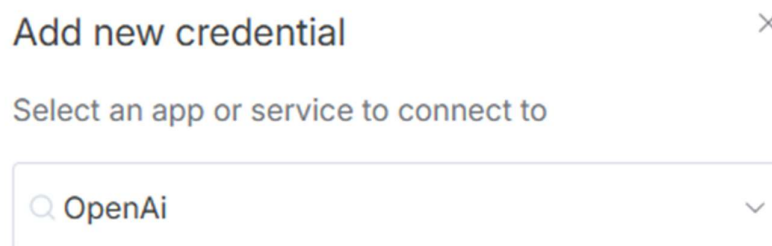
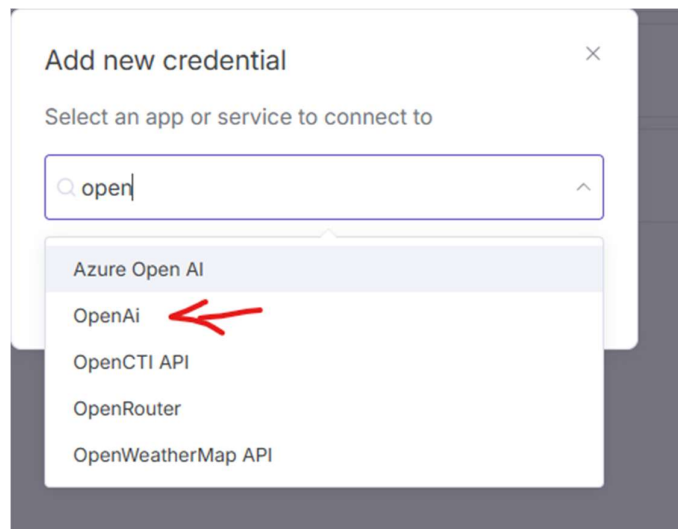
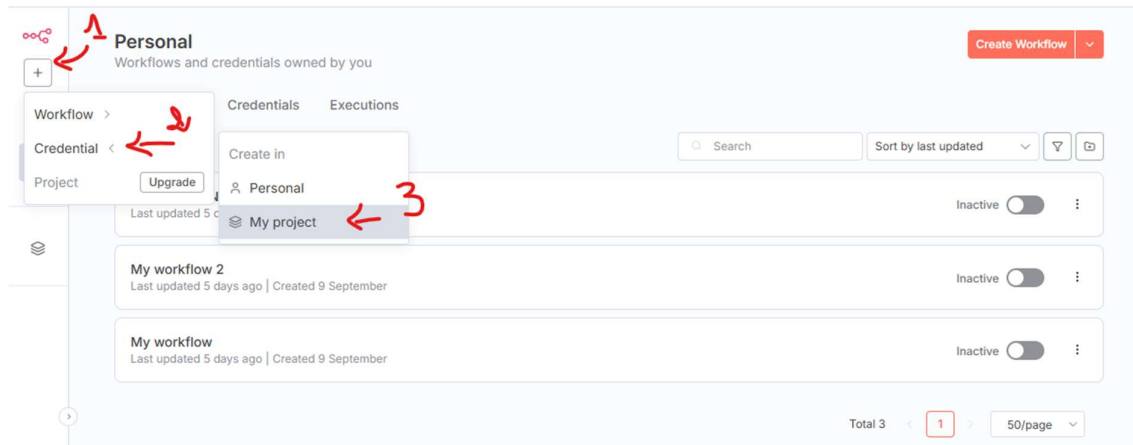
1. Make sure prerequisites are ready

- You have:
 - Top branch (train) working: /diabetes-upload
 - Bottom branch (predict) working: /diabetes-predict (returns prediction:0 or 1).
- You own an **OpenAI API key** (from <https://platform.openai.com> → API keys)(skip this part if you don't own an OpenAI API key, we can use a free OpenAI API for this labs provided by n8n)
- Copy and paste this link in your navigator: <https://platform.openai.com/api-keys> , log in to your account, go to dashboard, API keys and then click Create new secret key, save your key: The API key will appear, but it will only be shown to you once. Immediately copy this key and store it in a secure password manager or other safe place. You will not be able to view the full key again after closing the prompt





2. Add credentials in n8n Cloud (in our case we will use the n8n free open API, skip this part go to the next one)
1. In the editor, left menu → **Credentials** → + **New**.
2. Choose **OpenAI API**.
3. Paste your API key → **Save**.



OpenAi account
OpenAi

2 → Save ×

Connection

Need help filling out these fields? [Open docs](#)

Sharing

Ask Assistant for setup instructions

Details

1 →

API Key *

Organization ID (optional)

Only required if you belong to multiple organisations

Base URL

https://api.openai.com/v1

Allowed HTTP Request Domains

All

Enterprise plan users can pull in credentials from external vaults. [More info](#)

OpenAi account
OpenAi

Saved ×

Connection

→

Connection tested successfully

Retry

←

Sharing

Details

Need help filling out these fields? [Open docs](#)

Ask Assistant for setup instructions

API Key *

Organization ID (optional)

Only required if you belong to multiple organisations

Base URL

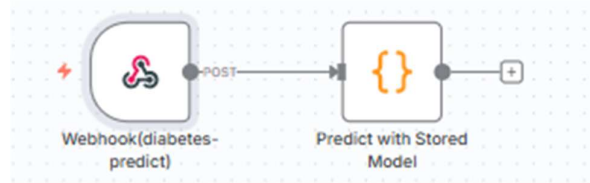
https://api.openai.com/v1

Allowed HTTP Request Domains

All

3. Extend the “predict” branch

Current flow (bottom branch): Webhook (diabetes-predict) -> Predict with Stored Model



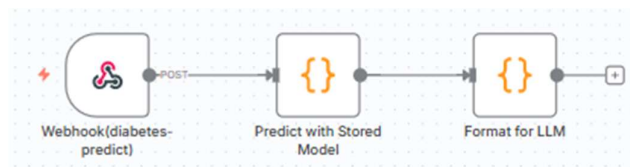
We'll add two nodes:

1. **Code** – Format a clean summary (optional but makes the prompt tidy)
2. **OpenAI Chat Model** – Generate explanation + recommendations

Step 3.1 – Code node (“Format for LLM”)

1. Drag a **Code** node after Predict with Stored Model.
2. Name it: Format for LLM.
3. Paste this:

```
// Extract info from the prediction node
const { usedK, featuresOrder, input, prediction } = $json;
const label = prediction ? "Diabetes: YES (1)" : "Diabetes: NO (0)";
const features = Object.fromEntries(featuresOrder.map(k => [k, input[k]]));
return [{
  json: {
    summary: { algorithm: "KNN", k: usedK },
    result: { value: prediction, label },
    features
  }
}];
```



Step 3.2 – Add OpenAI -> Message a model

1. Click + → search **OpenAI** → choose **Message a model**
2. **Credentials:** select `n8n freeAI API credits`
3. **Model:** `gpt-4o-mini`.
4. **Input / Prompt** (paste the script below):
5. **(Optional)** Temperature: **0.7** | Max tokens: **700**.

You are a careful clinician providing general lifestyle guidance.

You are NOT giving a diagnosis or treatment; always include a short disclaimer at the end.

Be concise, use plain language, and give 5–8 actionable bullet points.

If prediction = "Diabetes: YES (1)", explain likely risk and concrete next steps.

If prediction = "Diabetes: NO (0)", focus on prevention based on risk factors.

Tailor advice to the provided feature flags and numbers.

Context:

Model: `{{ $json.summary.algorithm }}` (`k={{ $json.summary.k }}`)

Prediction: `{{ $json.result.label }}`

Features (JSON):

`{{ JSON.stringify($json.features, null, 2) }}`

Tasks:

- 1) One short paragraph explaining the result.
- 2) 5–8 tailored recommendations (bullets).
- 3) A clear one-line disclaimer that this is educational info only, not medical advice.


Message a model
Execute step

Parameters
Settings
Docs

Credential to connect with
n8n free OpenAI API credits

Resource
Text

Operation
Message a Model

Model
From list GPT-4O-MINI

Messages

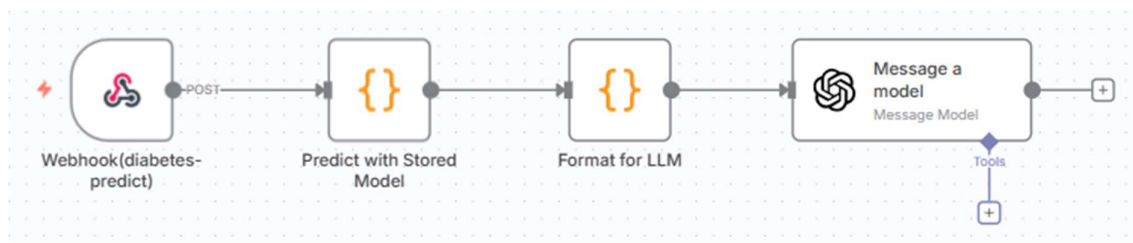
Prompt

```

You are a careful clinician providing general lifestyle guidance.
You are NOT giving a diagnosis or treatment; always include a short disclaimer at the end

```

Role
User



Step 3.3 – Make it response

- In your **Webhook (diabetes-predict)** node, set **Response = Last node**.
- Make sure **Message a model** is the **last node** in the predict branch.

Step 3.4 – Activate & Test

1. Click **Activate workflow** (so production URLs work all the time).



2. Send a test request:

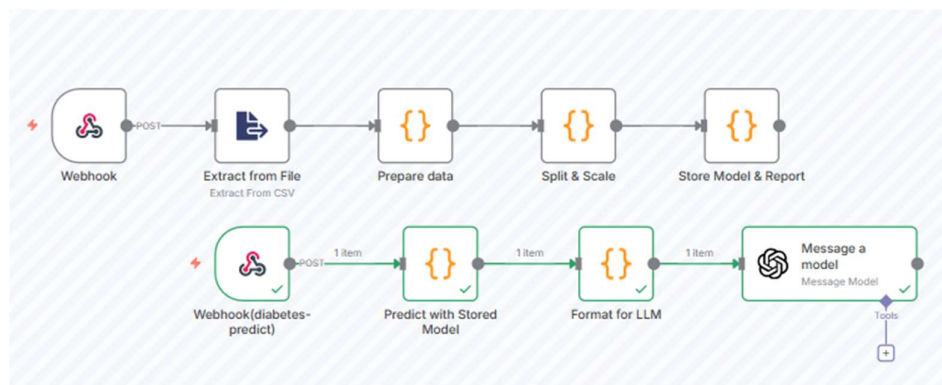
```
HP@NSAM MINGW64 ~
$ curl -X POST "https://ueltest.app.n8n.cloud/webhook/diabetes-predict" \
  -H "Content-Type: application/json" \
  -d '{"HighBP":1,"HighChol":1,"CholCheck":1,"BMI":30,"Smoker":0,"Stroke":0,"HeartDiseaseorAttack":0,"PhysActivi
ty":1,"Fruits":1,"Veggies":1,"HvyAlcoholConsump":0,"AnyHealthcare":1,"NoDocbcCost":0,"GenHlth":3,"MentHlth":2,"P
hysHlth":1,"DiffWalk":0,"Sex":0,"Age":9,"Education":4,"Income":6}'
```

Test Request

% Total	% Received	% Xferd	Average Speed	Time	Time	Time	Current
			Dload Upload	Total	Spent	Left	Speed
100	1735	0	1452 100	283	222	43	0:00:06
							0:00:06
							--:--:--
							338

```
{
  "index": 0,
  "message": {
    "role": "assistant",
    "content": "The prediction indicates that you are not currently at high risk for diabetes. This is a positive finding, but it's essential to continue focusing on healthy lifestyle choices to maintain this status and prevent any future risk. Here are some actionable recommendations to help you stay on track: \n\n- **Eat a Balanced Diet**: Focus on whole foods, including plenty of fruits, vegetables, whole grains, lean proteins, and healthy fats. Limit processed foods and added sugars. \n\n- **Stay Active**: Aim for at least 150 minutes of moderate exercise per week, such as brisk walking, cycling, or swimming. \n\n- **Maintain a Healthy Weight**: If you're at a healthy weight, continue to maintain it. If you need to lose weight, set realistic goals and make gradual changes. \n\n- **Monitor Portion Sizes**: Be mindful of portion sizes to help manage calorie intake and maintain a healthy weight. \n\n- **Stay Hydrated**: Drink plenty of water throughout the day and limit sugary drinks. \n\n- **Get Regular Check-ups**: Regular health check-ups can help catch any potential health issues early. \n\n- **Manage Stress**: Incorporate stress-reducing activities into your routine, such as yoga, meditation, or deep-breathing exercises. \n\nThis information is for educational purposes only and is not a substitute for professional medical advice."
  }
}
```

Result



3. Result

Sep 14, 23:57:57
Succeeded in 5.438s | ID#127
+ Add tag

Copy to editor

Workflow diagram showing the process: Webhook → Extract from File → Prepare data → Split & Score → Store Model & Report → Message a model.

Logs: Message a model: Success in 3.669s

Success in 5.438s

Webhook(diabetes-predict)

Predict with Stored Model

Format for LLM

Message a model

INPUT

summary

algorithm KNN

k 7

result

value 0

label Diabetes: NO (0)

features

OUTPUT

index 0

message

role assistant

content The prediction indicates that you are not currently at risk for diabetes. This is a positive outcome, but it's still important to maintain healthy habits to prevent potential risk factors in the future. Here are some actionable recommendations to help you stay healthy: **Eat balanced meals** Focus on whole foods like fruits, vegetables, whole grains, lean proteins, and healthy fats. **Stay active** Aim for at least 150 minutes of moderate exercise each week, such as brisk walking, cycling, or swimming. **Monitor your weight** Keeping a healthy weight can significantly reduce the risk...

refusal [null]

annotations

logprobs [null]

finish_reason stop